UT CHETUMAL

Desarrollo y Gestión de Software

Extracción de conocimiento en bases de datos

Dr. Martín Antonio Santos Romero

T.S.U. Miguel Ángel Peña Pech

9no cuatrimestre

Práctica autónoma 1

Primeros pasos con Anaconda-Jupyter-Python

Índice

1. Introducción
2. Instalar librerías en Python
2.1 ¿Qué es una librería en Python?
2.2 Principales librerías de Python
- NumPy
- Pandas
- Matplotlib
- Scikit-learn
- TensorFlow
- Keras
- PyTorch

Tabla 1. Características de librerías de Python
Figura 1. Instalación de Pandas
3. Importar librerías/paquetes en Python
Figura 2. Importación de Pandas
4. Lectura de datos con Pandas
4.1 Lectura de archivo CSV
4.2 Vista preliminar del DataFrame
Figura 3. Vista inicial del DataSet
5. Manipulación de datos
5.1 Ordenamiento por columna
Figura 4. DataSet ordenado por esperanza de vida
6. Visualización de datos con Matplotlib
6.1 Instalación de Matplotlib
Figura 5. Instalación de Matplotlib
6.2 Gráfica de dispersión simple
Figura 6. Renta per cápita vs Esperanza de vida
7. Personalización de gráficos
7.1 Agregar título y etiquetas a los ejes
Figura 7. Gráfica con título y ejes
8. Gráfica comparativa avanzada
8.1 Escalado por población
8.2 Anotaciones y ajuste visual
Figura 8 y 9. Gráfica comparativa por país
9. Conclusión

Introducción

En el mundo del análisis de datos y la inteligencia artificial, Python se ha convertido en una herramienta fundamental, esta práctica tiene como objetivo introducirnos como estudiante en el desarrollo de Python utilizando herramientas como **Anaconda** y **Jupyter Notebook**. A través de una serie de ejercicios prácticos, se pretende enseñar cómo instalar librerías, manipular datos con **pandas**, y visualizar información de forma gráfica con **matplotlib**.

1: Instalar librerías en Python

-Define qué es una librería en Python

Una librería en Python es un conjunto de módulos que contiene funciones, clases y variables predefinidas que permiten realizar tareas específicas sin tener que programarlas desde cero. Estas librerías ayudan a extender las capacidades del lenguaje, permitiendo a los desarrolladores resolver problemas complejos de manera más eficiente.

-Define y caracteriza las principales librerías de Python: Numpy, Pandas, Matplotlib, Scikit- learn, Tensorflow, Keras, Pytorch, etc).

Tabla 1. Características de librerías de Python.

NumPy	
Definición	Significa **Numerical Python**. Es una librería fundamental para realizar operaciones numéricas y científicas en Python.
Características	• Soporte para arreglos (arrays) multidimensionales. • Funciones matemáticas avanzadas. • Excelente rendimiento para cálculos vectorizados.
Pandas	
Definición	Proporciona estructuras de datos de alto nivel como **Series** y **DataFrame**, ideales para análisis de datos.
Características	• Manipulación eficiente de datos estructurados (filas y columnas). • Lectura y escritura de archivos CSV, Excel, JSON, SQL, etc. • Herramientas para limpieza, filtrado, agrupamiento y análisis de datos.
Matplotlib	
Definición	Librería para la **visualización de datos en gráficos** estáticos, animados e interactivos.
Características	• Creación de gráficos de líneas, barras, dispersión, histogramas y más. • Personalización de estilos, títulos y etiquetas. • Compatible con Jupyter Notebooks.
Scikit-learn	
Definición	Librería especializada en **aprendizaje automático**.
Características	• Incluye algoritmos de clasificación, regresión, clustering y reducción de dimensionalidad. • Fácil de usar e integrar con otras librerías como NumPy y Pandas. • Ideal para tareas de machine learning supervisado y no supervisado.
TensorFlow	
Definición	Framework de código abierto desarrollado por Google para **construir y entrenar modelos de aprendizaje profundo** (deep learning).
Características	• Alta escalabilidad en GPU y TPU.

	• Soporta redes neuronales complejas y aprendizaje automático avanzado. • Compatible con múltiples lenguajes, aunque principalmente usado con Python.
Keras	
Definición	Librería de alto nivel construida sobre **TensorFlow** que permite crear redes neuronales de forma más sencilla.
Características	• API intuitiva y fácil de usar. • Soporta modelos secuenciales y funcionales. • Ideal para prototipos rápidos de redes neuronales.
PyTorch	
Definición	**Framework de deep learning** desarrollado por Facebook.
Características	• Muy flexible y dinámico (define los modelos en tiempo de ejecución). • Amplia adopción en la comunidad investigadora. • Soporta entrenamiento en GPU y construcción de redes neuronales complejas.

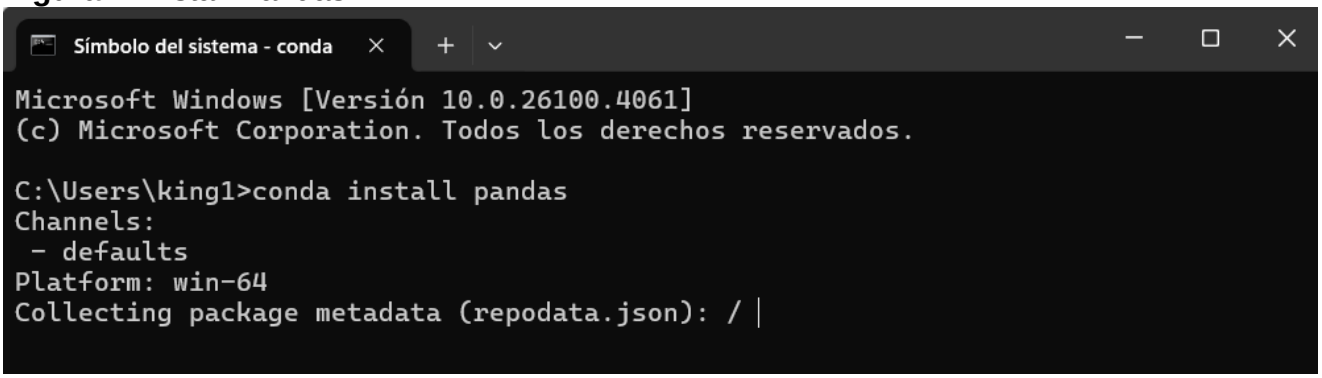
Fuente: Angel Peña (2025).

Para instalar una librería se utiliza el comando de gestión de paquetes conda:

conda install <libreria>

Instala la librería Pandas.

Figura 1. Install Pandas.



Fuente: Angel Peña (2025).

2: Importar librerías/paquetes en Python:

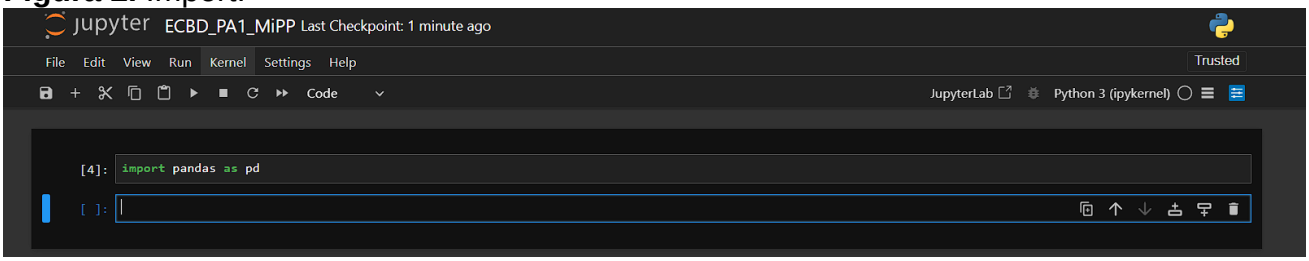
Para importar un paquete se usa la instrucción:

import <librería> as <nombre>

Por ejemplo, para importar Pandas:

import Pandas as pd

Figura 2. Import.



Fuente: Angel Peña (2025).

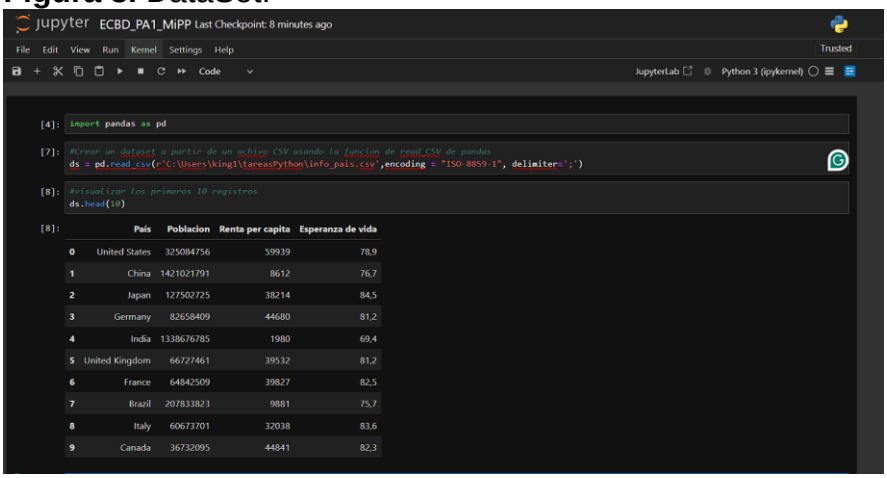
3: Realiza el siguiente código para importar la librería pandas y una lectura de una fuente de datos.

- Guarda tu código con el nombre de ECBD_PA1_XXYZ. Sustituye tus iniciales.
- Utiliza el archivo anexo Info_pais.csv

```
import pandas as pd
ds = pd.read_csv(r'C:\Users\msantos\Programas Python\Info_pais.csv',encoding = "ISO-8859-1",delimiter=';')

ds.head(10)
```

Figura 3. DataSet.



Fuente: Angel Peña (2025).

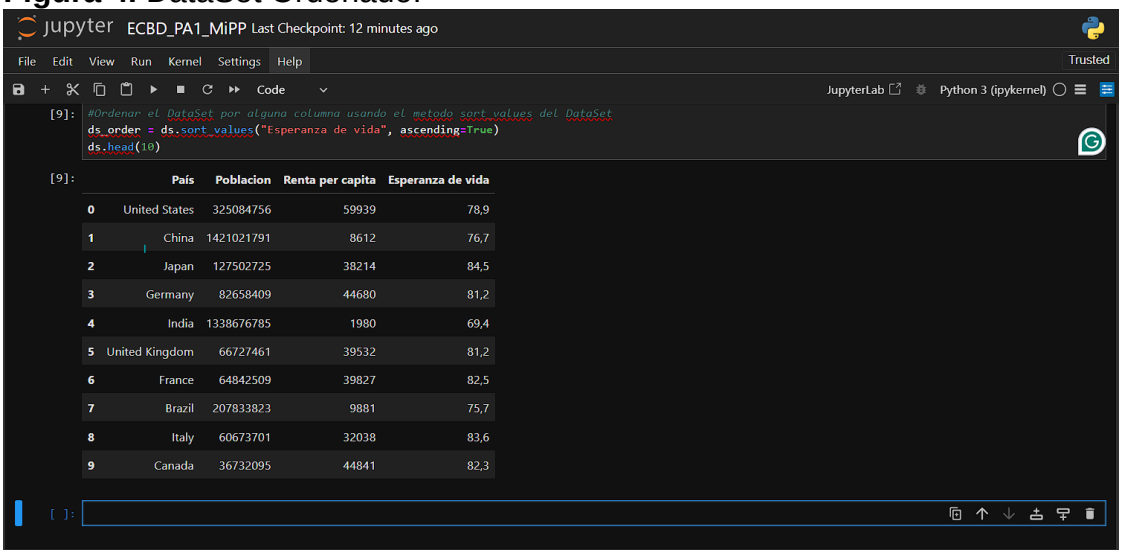
***** Describe para qué sirve cada método y las funciones que utilizaste***

Primero, importamos la librería utilizando un alias **(pd)** para facilitar su uso en el resto del programa, después empleamos la función **read_csv()** para leer el archivo en formato **CSV** que contiene información de distintos países. Esta función permite especificar la ruta del archivo, el tipo de codificación (**ISO-8859-1**, comúnmente usada en archivos con caracteres en español) y el delimitador de columnas, que en este caso es el punto y coma **(;)**.

4: Realiza el siguiente código para manipular los datos, en este caso, ordenarlos por alguna columna.

```
ds_order = ds.sort_values("Esperanza de vida",ascending=True)
ds_order.head(10)
```

Figura 4. DataSet Ordenado.



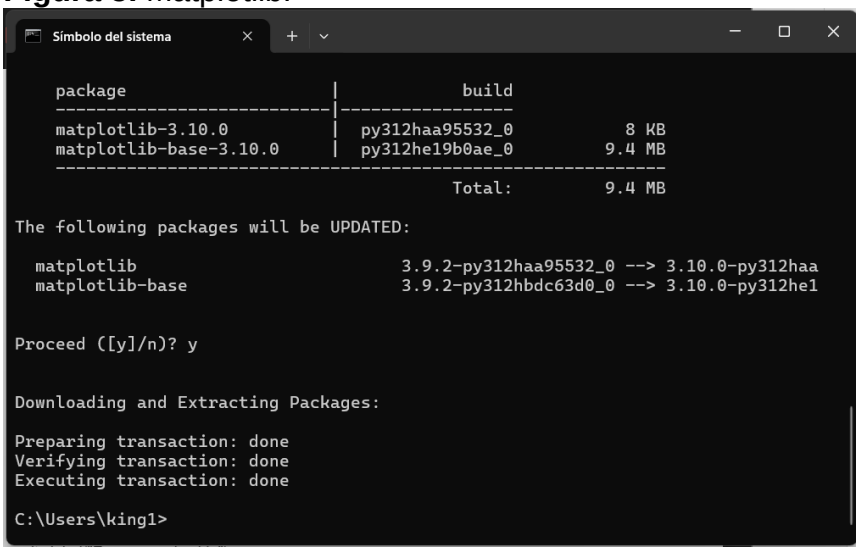
Fuente: Angel Peña (2025).

***** Describe para qué sirve cada método y las funciones que utilizaste.***

Primero, utilizamos el método **sort_values()** para ordenar el conjunto de datos **(ds)** en función de la columna "Esperanza de vida", estableciendo el parámetro **ascending=True** para que el orden sea de menor a mayor. Esta operación permite reorganizar las filas del DataFrame según los valores de esa columna específica. El resultado se almacena en una nueva variable llamada **ds_order**, que contiene los datos ya ordenados. Después, usamos el método **head(10)** para mostrar las primeras diez filas del DataFrame ordenado, lo que permite visualizar los países con menor esperanza de vida en los datos analizados.

5: Instala la librería Matplotlib.

Figura 5. Matplotlib.

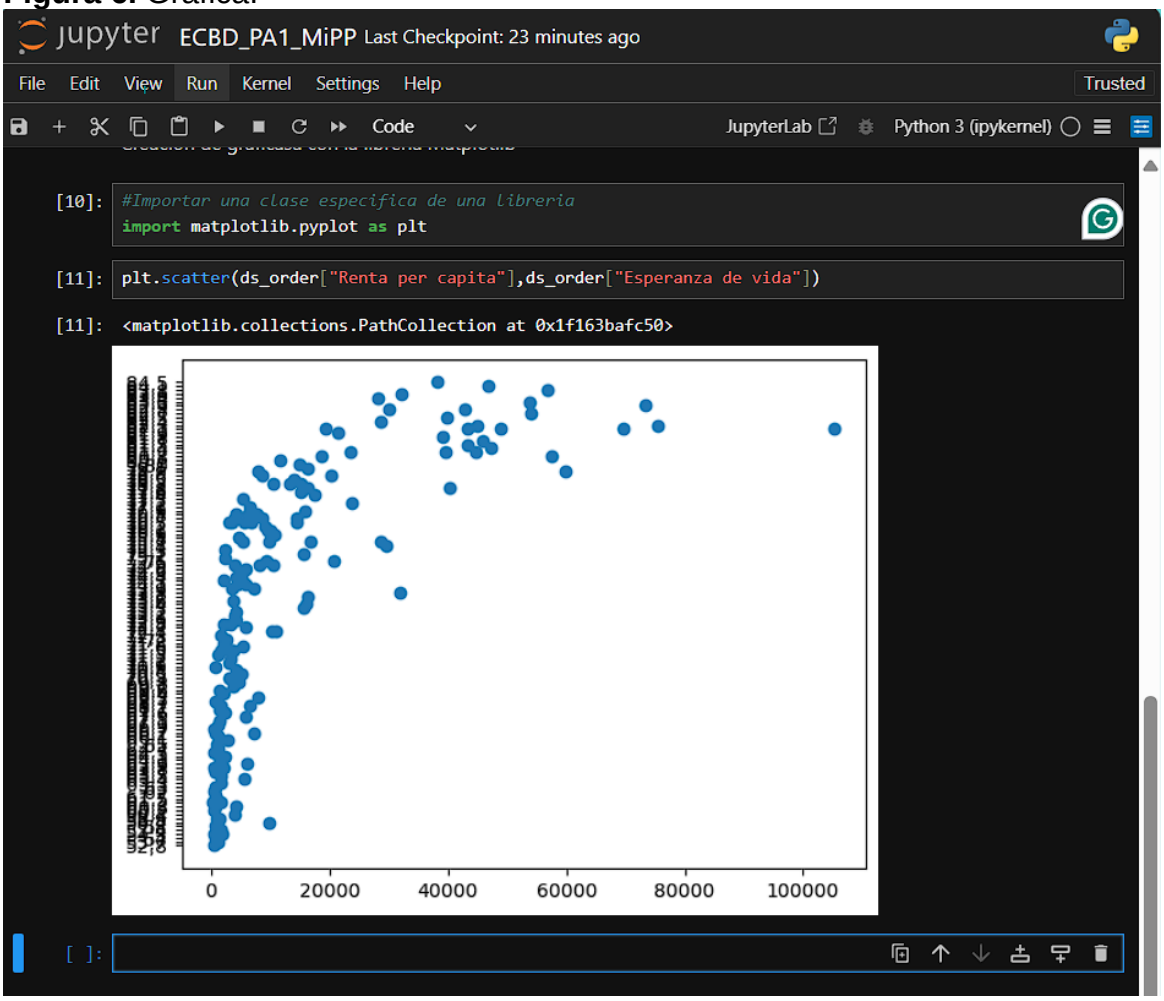


Fuente: Angel Peña (2025).

6: Realiza el siguiente código para visualizar datos usando Matplotlib, en este caso a través de una gráfica de dispersión.

```
import matplotlib.pyplot as plt
plt.scatter(ds_order["Renta per capita"],ds_order["Esperanza de vida"])
```

Figura 6. Gráfica.



Fuente: Angel Peña (2025).

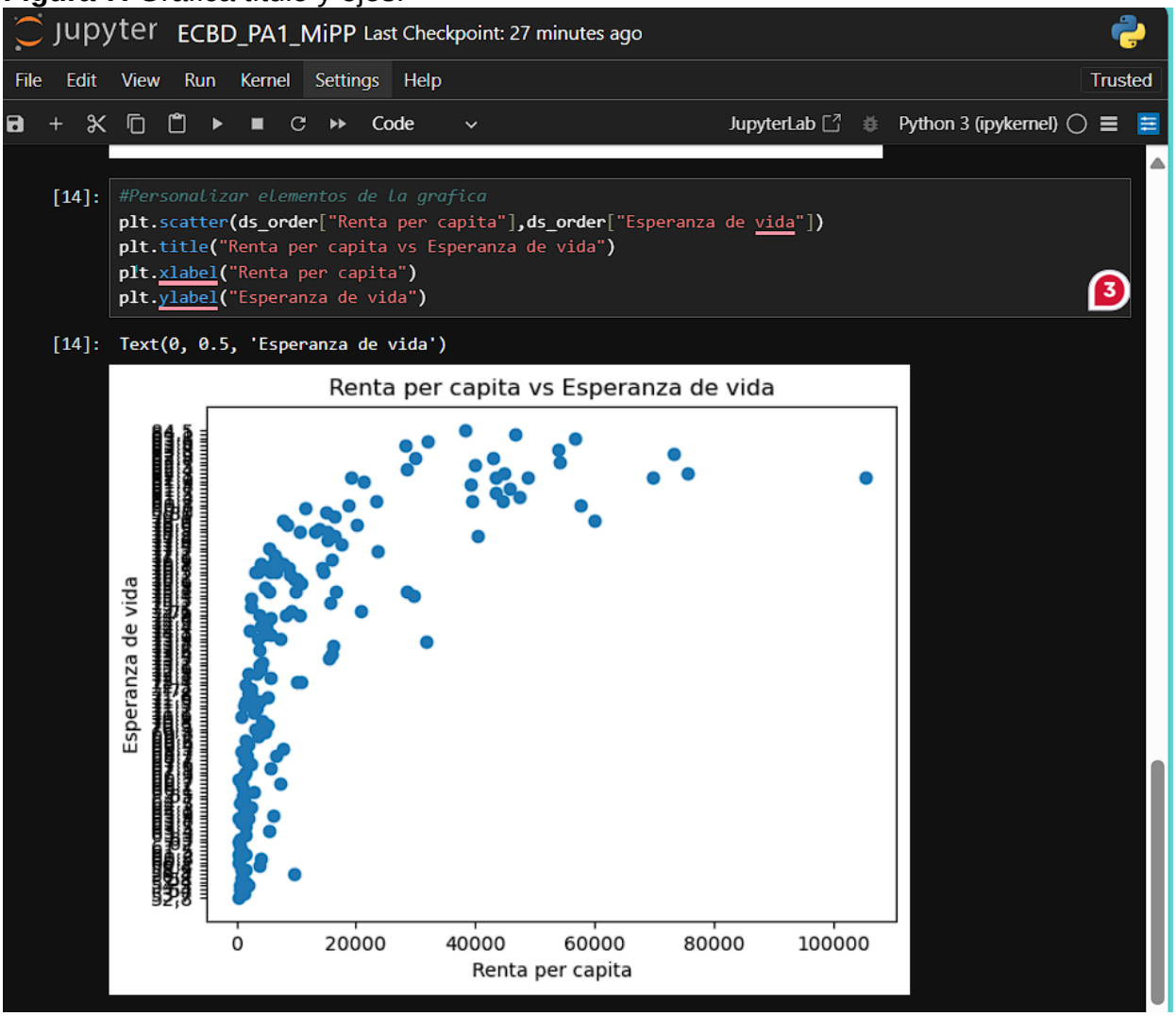
***** Describe para qué sirve cada método y las funciones que utilizaste.***

Importamos el módulo **pyplot** de la librería **matplotlib** utilizando el alias **plt**, lo cual facilita la creación de gráficos de manera sencilla. A continuación, se utiliza la función **scatter()** para generar una gráfica de dispersión que representa la relación entre dos variables: "Renta per cápita" en el eje X y "Esperanza de vida" en el eje Y.

7: Agregar títulos a los ejes de la gráfica utilizando el siguiente código:

```
plt.scatter(ds_order["Renta per capita"],ds_order["Esperanza de vida"])
plt.title("Renta per capita vs Esperanza de vida")
plt.xlabel("Renta per capita")
plt.ylabel("Esperanza de vida")
```

Figura 7. Gráfica título y ejes.



Fuente: Angel Peña (2025).

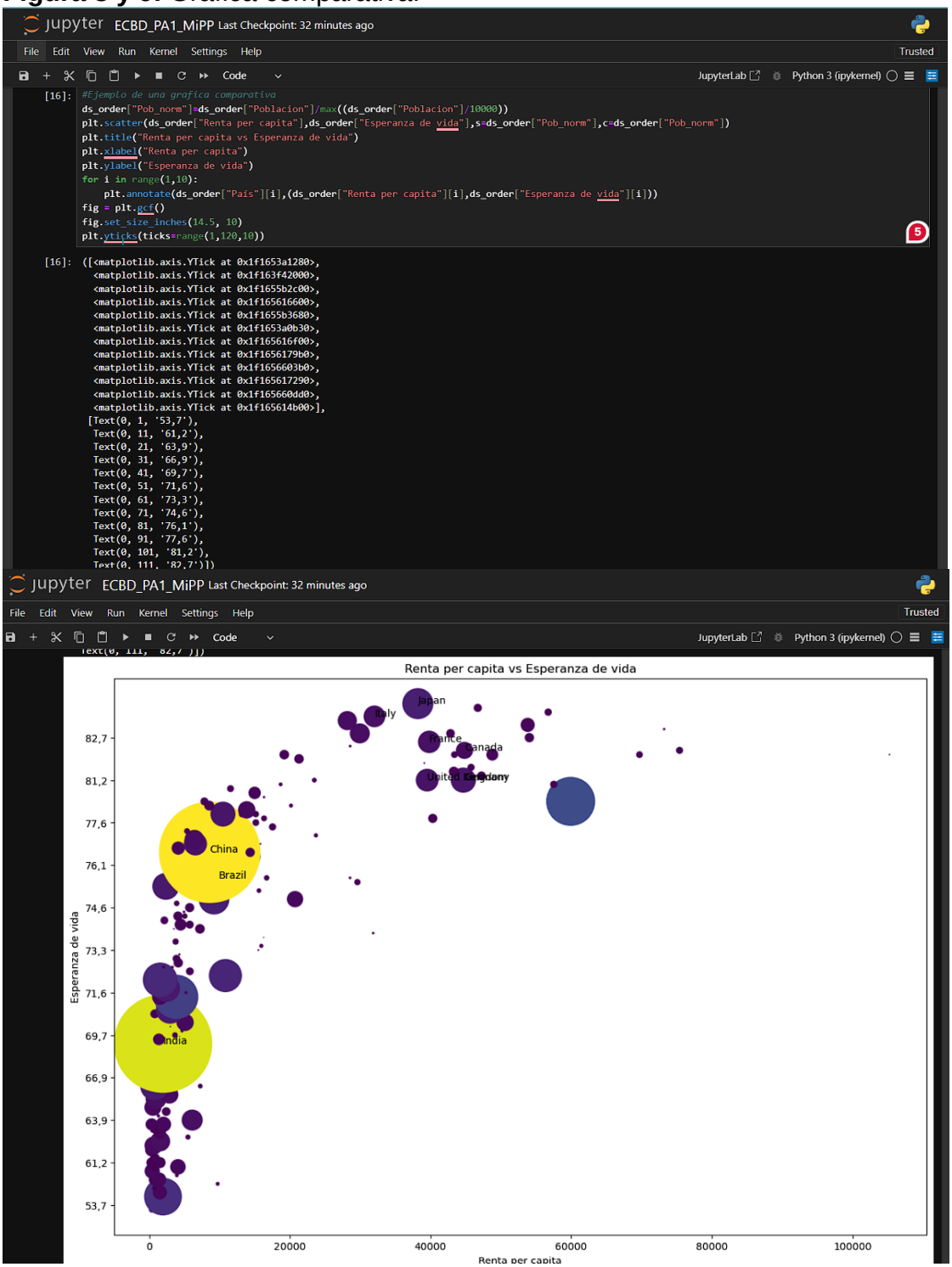
***** Describe para qué sirve cada método y las funciones que utilizaste.***

A la gráfica anterior se le anexó ejes y titulo

8: Prueba el siguiente código de una gráfica comparativa de los datos:

```
ds_order["Pob_norm"]=ds_order["Poblacion"]/max((ds_order["Poblacion"]/10000))
plt.scatter(ds_order["Renta per capita"],ds_order["Esperanza de
vida"],s=ds_order["Pob_norm"],c=ds_order["Pob_norm"])
plt.title("Renta per capita vs Esperanza de vida")
plt.xlabel("Renta per capita")
plt.ylabel("Esperanza de vida")
for i in range(1,10):
plt.annotate(ds_order["País"][i],(ds_order["Renta per
capita"][i],ds_order["Esperanza de
vida"][i]))
fig = plt.gcf()
fig.set_size_inches(14.5, 10)
plt.yticks(ticks=range(1,120,10))
```

Figura 8 y 9. Gráfica comparativa.



Fuente: Angel Peña (2025).

*** Describe para qué sirve cada método y las funciones que utilizaste.*

primero se normalizó los valores de la población creando una nueva columna llamada "**Pob_norm**", dividiendo cada valor entre el máximo, con el fin de utilizar esta información para escalar el tamaño de los puntos en la gráfica. Luego, se utilizó **plt.scatter()** para generar una gráfica de dispersión que muestra la relación entre la renta per cápita y la esperanza de vida de los países.

Conclusión

Esta práctica me permitió familiarizarme con el entorno de desarrollo **Anaconda-Jupyter** y aprender los primeros pasos en el manejo de datos con **Python**. Se logró instalar e importar librerías clave, realizar lecturas y manipulaciones de datos desde archivos CSV, así como representarlos gráficamente. Estas habilidades son esenciales para comenzar a explorar el análisis de datos de manera estructurada, permitiendo transformar información cruda en conocimiento útil.